Reproducing and Extending PID-Comm in Software: Hypercube Collectives, Host-Side Cost Model, and BFS on a Simulated Processing-in-DIMM System

Rishikesh Machamalla
Dept. of CSE, IIT Guwahati
Guwahati, India
230101060

Aditya Mogili
Dept. of CSE, IIT Guwahati
Guwahati, India
230101065

Roopesh Jarupula
Dept. of CSE, IIT Guwahati
Guwahati, India
230101048

***Abstract*—PID-Comm (ISCA 2024) speeds up inter-PE collective communication on commodity UPMEM processing-in-DIMM devices by moving data modulation from the host onto the PEs, into vector registers, and across the domain-transfer boundary. We had no PIM hardware, no AVX-512 CPU, and could not run the authors' code on the UPMEM simulator. We therefore built a bit-accurate software model of the entangled-group memory layout and domain transfer, implemented the virtual hypercube and four primitives (AlltoAll, ReduceScatter, AllGather, AllReduce) at the paper's four implementation levels, verified them against an independent NumPy reference (17,976 C++ cases; 334 Python tests), and measured the *host-side* cost on one laptop. Trends agree with the paper for data size, PE count, element width and shape sensitivity, while magnitudes differ (e.g. ReduceScatter 1.38× vs. 4.46×). We add (i) an analytical host-cost model with exact event counts and fitted per-event costs (15.4% error on held-out hypercube shapes, 10.5% on an unseen BFS application), which does *not* beat a trivial rule at choosing the best level; (ii) the observation that bitwise reductions need no domain transfer; and (iii) a negative-then-fixed study of AVX2 kernels. All numbers are host runtimes of a software model, not UPMEM measurements.**

***Index Terms*—processing-in-memory, UPMEM, collective communication, hypercube, performance modeling, DRAM**

I. Introduction

Commodity PIM-enabled DIMMs such as UPMEM attach a processing element (PE) to each DRAM bank, but PEs have no direct links, so inter-PE communication goes through the host CPU. The host is slow at it because of three costs identified in the paper [1]: the host reorders data on its own, stages it in host memory, and performs a *domain transfer* (DT) on everything it touches. PID-Comm abstracts the PEs as a multi-dimensional *virtual hypercube*, lets users run many collectives at once over chosen axes, and removes the three costs with PE-assisted reordering (PR), in-register modulation (IM) and cross-domain modulation (CM).

Scope. Following the assignment, we reproduce the core idea at reduced scale (8–64 simulated PEs, four of the eight primitives, one application), compare trends with the paper's figures, and add something new. Real-hardware experiments are excluded. **Contributions:** (1) a software model and implementation of the hypercube and four primitives at four levels with an independent correctness oracle; (2) a trend comparison with the paper; (3) a BFS application; (4) a validated analytical host-cost model; (5) two further findings on bitwise reductions and SIMD kernels.

II. The Paper's Idea in Brief

Entangled groups and domain transfer. A rank has eight chips; a 64-bit word is split into eight bytes, one per chip, so the eight PEs at the same position form an *entangled group* that must be accessed together to use the full 64-byte burst. Data is stored in the PE domain, so the host cannot interpret it without an 8×8 byte transpose (DT).

Virtual hypercube. Users define dimensions (e.g. $4 \times 2 \times 4$) and a mask choosing the axes of each communication group (e.g. "101" selects x and z); every group runs its own collective at the same time. All dimensions but the last are powers of two, and groups have at least eight PEs, which guarantees that an entangled group always moves as a whole.

Three techniques. PR: PEs reorder their data locally so the host only moves whole blocks. IM: the host streams blocks through vector registers instead of host memory. CM: where no arithmetic is needed, the DT and the word shift fuse into one byte-level shift in the PE domain.

III. Implementation and Setup

A. Why not the authors' code

We cloned the authors' repository, which bundles a modified UPMEM SDK containing the functional simulator. After compatibility fixes (missing shared-library version links and `libtinfo.so.5`) the tutorial compiled and linked against the simulator backend. Execution then crashed with a segmentation fault inside `dpu_rank_handler_get_rank`, both with the authors' `dpu_alloc_comm` and the standard `dpu_alloc`. The allocator appears to contain machine-specific rank logic; we did *not* establish the root cause, and we did not obtain a full official SDK. Their host code also

uses AVX-512, which our CPU lacks. We therefore did not use the authors' implementation for any result.

B. Software model

Each entangled group's memory is stored in the bus layout: byte i of PE c's word is at burst offset $8i + c$. A host bus access is a 64-byte copy; PE-side access gathers bytes; DT is the 8×8 byte transpose. A routing plan assigns each PE's chunks to destination groups with a lane rotation so that every 64-byte block moves as a unit. We implement the hypercube (`groups(mask)`), AlltoAll, AllGather, ReduceScatter and AllReduce (including the fused AllReduce that avoids a round trip through PE memory) at four levels: **L0** baseline (host memory, DT, elementwise rearrangement), **L1** +PR, **L2** +IM, **L3** +CM. The host loops exist in scalar and AVX2 forms. The code is C++17 (measurements) plus a Python package (semantics, tests, experiments).

C. Correctness

An independent NumPy reference implements the four primitives from their definitions (integer sums wrap at the element width). The C++ test suite checks every primitive, level, hypercube shape/mask, element width (8/32/64 bit), operator (sum/or/xor) and kernel variant: **17,976 cases pass** (2,568 shape/mask combinations are correctly rejected as groups smaller than eight PEs). Python tests (**334 passed, 2 skipped**, the skipped ones needing AVX-512) include a word-for-word cross-check of the C++ backend against the reference; deliberately corrupting the lane rotation made 37 of the 200 cross-check tests fail, so the cross-check is not vacuous.

D. Measurement protocol

Host: Intel Core i7-1360P (AVX2, no AVX-512), Fedora 43, GCC 15.2.1, power profile *performance*, process pinned to one performance core, one warm-up repetition then the minimum of 15. The minimum of 15 pinned repetitions was chosen after measuring the noise floor (same 48 configurations measured twice): unpinned median-of-5 gave 18% median run-to-run difference, pinned minimum gave **2.2% mean** (worst 13.3%). Reported times are *host-side* time only; the simulated bus is a memory copy and PE-side work is simulated on the CPU and excluded. **Absolute numbers are therefore not comparable with the paper; only ratios and trends are.** The main comparison uses identical scalar kernels at every level so that the algorithm is the only variable; AVX2 is reported separately.

IV. Baseline Reproduction vs. the Paper

Table I compares our ratios (64 PEs, 64 KiB per PE, 32-bit sums, scalar kernels) with the numbers reported in the paper.

Agreement. The speedup grows with data size (512 B → 64 KiB per PE) and with PE count (8→64), 8-bit data benefits most because CM then applies to reductions, and AlltoAll/AllReduce throughput is nearly flat across shapes. **Disagreement.** AlltoAll gains more than in the paper; ReduceScatter and AllReduce gain less; PE-assisted reordering *alone* makes our host code slower (0.67×), whereas the paper reports 1.48×; the CM step is larger than reported. We did not test the causes. Plausible, untested explanations: our "bus" is a memory copy so the baseline's host-memory staging is cheaper relative to real DIMM traffic; our PE-side reordering work is excluded from host time, so PR shows only its cost on the host side; and our baseline reduction is a simple streaming loop. We therefore make *no* claim to reproduce the paper's magnitudes. The shape experiment (3-D configurations) is only partly comparable: our shapes differ from the paper's, and the paper's ReduceScatter/AllGather growth with x-axis length is not clearly reproduced (max/min 1.21 and 1.12).

TABLE I
Our host-side ratios vs. paper (real hardware, 1024 PEs).

Quantity (full design vs. baseline)	Paper	Ours
AlltoAll speedup	5.19×	9.53×
ReduceScatter speedup	4.46×	1.38×
AllReduce speedup	4.23×	3.32×
AllGather speedup	—	2.39×
+PR step (geomean)	1.48×	0.67×
+IM step (additional)	2.03×	2.46×
+CM step (AA/AG)	1.42×	3.15×
Speedup with data size (↑)	yes	2.49→3.19×
Speedup with #PEs (↑)	yes	1.79→3.23×
8-bit benefits most (RS/AR)	yes	2.88 / 2.13 / 1.86×
AA/AR flat over shapes (max/min)	yes	1.08 / 1.20

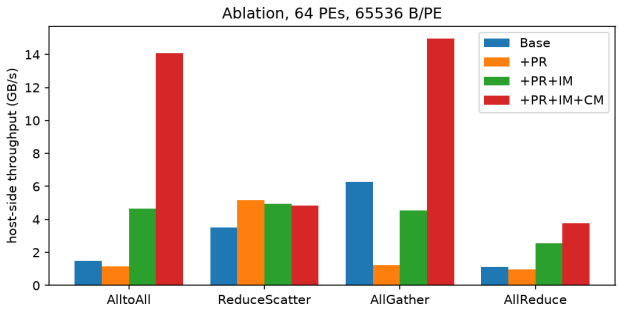


Fig. 1. Ablation: host-side throughput of the four levels (64 PEs, scalar kernels).

V. Application: Breadth-First Search

BFS partitions edges over PEs; each round every PE expands the frontier over its edges, then AllReduce with bitwise OR merges the next frontier, as in the paper. We use a random undirected graph (V=16,384, E=65,532; 16,380 vertices reachable; 8 rounds). Correctness is checked three ways: a Python version using the reference semantics, the C++ version on identical graph files (reached count and distance sum agree, same number of rounds), and a plain queue BFS, including a deep path graph and a disconnected graph in the tests. Only the AllReduce is timed: PE kernel time is unavailable without hardware, so we claim **no end-to-end speedup**. The communication speedup grows with PE count, as in the paper's scaling discussion, but our 2.0–2.5× is a

TABLE II
BFS COMMUNICATION (HOST SIDE, MS, 8 ROUNDS), SCALAR KERNELS.

PEs	L0	L1	L2	L3	meas.	model
8	0.038	0.068	0.051	0.019	2.03×	2.02×
16	0.075	0.132	0.098	0.036	2.06×	1.99×
32	0.179	0.294	0.195	0.074	2.41×	1.97×
64	0.386	0.613	0.401	0.157	2.46×	1.96×

TABLE III
COST-MODEL VALIDATION (MEAN ABS. ERROR UNLESS NOTED).

	Scalar	AVX2
In-sample (856 rows)	14.9%	14.3%
Leave-one-config-out	15.4%	14.5%
Size extrapolation (≤2K→≥8K)	18.1%	17.2%
BFS (unseen application)	10.5%	9.1%
Speedup L0/L3 prediction error	26.2%	21.6%
Measurement noise floor	2.2%	2.7%
Level choice correct	80.4%	80.8%
"Always L3" rule correct	80%	81%

communication-only host measurement and is not comparable with the paper's 1.20–3.99× end-to-end application range. Level 1 is slower than the baseline for the same reason as in Table I. At L3 BFS needs *zero* domain transfers (Sec. VI-B).

VI. NEW CONTRIBUTIONS

A. Analytical host-cost model

Design. The cost of a primitive at a level is $T = \sum_k c_k n_k$ over seven event counts n_k (bus bursts, domain transfers, host-memory blocks, rotates, lane permutes, combines, scalar word operations). The counts are closed-form functions of (primitive, level, #PEs, group size, bytes, element width, operator). The coefficients c_k are fitted by non-negative least squares on relative error. The two layers are validated separately.

Counts are exact. The formulas equal the C++ backend's own counters on **856 of 856** measured rows (and in a separate test over six shapes, four dtype/operator pairs, all primitives and levels).

Time prediction. Table III gives out-of-sample errors: hold out each hypercube shape/mask in turn (10 configurations), and train on small sizes (≤2 KiB) to predict large ones (≥8 KiB). The BFS application never appears in the training data.

Findings, including negative ones. (1) The model predicts runtime to about 15% on unseen shapes and 10% on BFS, four to seven times the measurement noise, so it is useful for estimating cost but is not exact. (2) **It is no better than a trivial rule at choosing the fastest level**: "always level 3" is correct in 80% of 224 decisions, as is the model. (3) Errors are not uniform: 22–24% for L0/L1 but about 9% for L2/L3; and for runs of ≥1 ms the error is about 54%. All eight worst predictions are level 1 at 64 PEs and 32 KiB per PE, measured near 1.6 ms against 0.6 ms predicted. We suspect the staging buffers exceed the cache; a cache-spill term and a per-call overhead term did not improve cross-validated error (nested selection, 15.4% either way), so the

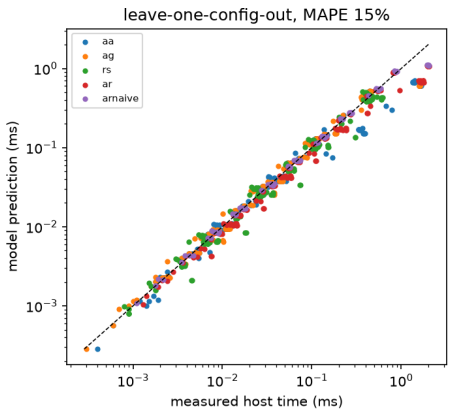


Fig. 2. Predicted vs. measured host time on held-out hypercube configurations.

cause remains unexplained. (4) The model underestimates how the BFS speedup grows with PEs, predicting a constant 2.0× (measured 2.03→2.46×). (5) Under AVX2 the fitted coefficients for combines and permutes are zero; this means the data cannot separate them from other terms, not that they are free.

B. Bitwise reductions need no domain transfer

The paper applies cross-domain modulation only to non-arithmetic primitives and to 8-bit data. OR/XOR (used by BFS) are independent of the byte layout, so at L3 they can skip the domain transfer too. Measured at 64 PEs on 8-byte elements, ReduceScatter with OR at L3 is **1.60**× and AllReduce **1.89**× faster than the same primitive with a sum at L2; BFS performs no domain transfers at L3 (Table II reports the resulting times). Correctness of all OR/XOR cases is covered by the test suite. This is a software observation on our model; we have not shown it on real hardware.

C. Host SIMD kernels: a falsified hypothesis and a fix

The paper's host code needs AVX-512, which our CPU lacks, so we implemented AVX2 kernels. Initially they were slower than scalar in the real data path, even though the kernel itself is faster in isolation. We first hypothesised that per-call overhead (function-pointer dispatch, one 64-byte block per call) was responsible; a microbenchmark *refuted* this: dispatch through a function pointer costs little (rotate of one block: 6.4 ns scalar, 1.5 ns AVX2 via pointer, 1.3 ns inlined — pointer and inlined AVX2 differ by only about 15%). A second microbenchmark of the real data path (copy in, kernel, copy out) showed the AVX2 kernel at 6.7 ns versus scalar 3.1 ns — more than twice as slow — falling to 2.6 ns once the whole loop is compiled for AVX2. We attribute this to the surrounding copies being compiled for baseline x86-64, so a wide load follows narrow stores; this fits the measurements but we did not verify the mechanism at the CPU level. Compiling each host loop per instruction set (identical correctness; 17,976 cases pass) made the AVX2 build 1.8×

(scalar run) and 1.4× (AVX2 run) faster than the pointer-dispatch AVX2 build. **However, the benefit over scalar is not reliable on this laptop**: the same experiment gave AVX2 vs. scalar of 1.06× in one run and 0.81× in the other (the AVX2 geomean differed by 33% between runs while scalar differed by about 1%). The most defensible conclusion is that inlining recovers an avoidable loss; AVX2 does not give a dependable speedup over scalar here, possibly because the kernel is a small share of total time next to the domain-transfer transpose and memory traffic (untested).

VII. Limitations and What We Left Out

Left out (as the assignment specifies): real UPMEM hardware and the authors' SDK/driver (we could not run it), thousands of PEs, the other four primitives, and the DLRM and GNN applications. **Substitutions:** a software model of the memory layout and domain transfer; host-only timing; BFS communication only. **Limitations:** (i) host timings reflect a laptop memory hierarchy and a memcpy "bus", not DIMM channels; (ii) PE-side time is not modelled, which penalises L1; (iii) at most 64 PEs, so PE-count scaling is only indicated; (iv) the entangled group is eight consecutive PE ids rather than the paper's full chip/bank/rank/channel mapping; (v) the cost model's remaining 15% error and the unexplained large-size level-1 misses; (vi) the model does not improve level selection over a trivial rule; (vii) the AVX2 result is unstable between runs; (viii) the hypercube requires groups of at least eight PEs and powers-of-two dimensions except the last, as in the paper; (ix) no AVX-512 measurements were possible on our machine.

VIII. Conclusion

A software model of PID-Comm reproduces the paper's trends in data size, PE count and element width, but not its magnitudes for reductions, and shows that PE-assisted reordering alone is not a win when only host time is counted. Our host-cost model predicts runtime to about 15% with exact event counts, but does not improve on always choosing the full design. Bitwise reductions can skip the domain transfer, and SIMD kernels must be compiled together with their surrounding loops to avoid losing their advantage. Real-hardware validation is the obvious next step.

Acknowledgment

We thank the course staff of CS5290 (Memory-Centric Architectures) for the reduced-scope assignment design, and the PID-Comm authors for making their paper and source repository publicly available.

Member Contributions

All three members read the PID-Comm paper and discussed the overall approach, scope reduction, and software-model strategy together. Rishikesh Machamalla set up the development environment, attempted and diagnosed the failure of the authors' reference implementation on the UPMEM functional simulator, implemented and ran the C++ and Python software model, executed the full correctness and measurement pipeline (including the scalar and AVX2 evaluation runs), and investigated and fixed the AVX2 performance regression; he is finalizing the report, LaTeX source, and source-code submission. Aditya Mogili prepared the Baseline Results document and performed a proofreading and formatting pass on the final report. Roopesh Jarupula prepared the presentation slides and the Contribution Results document.

References

[1] S. U. Noh, J. Hong, C. Lim, S. Park, J. Kim, H. Kim, Y. Kim, and J. Lee, "PID-Comm: A fast and flexible collective communication framework for commodity processing-in-DIMM devices," in *Proc. ISCA*, 2024.

[2] F. Devaux, "The true processing in memory accelerator," in *Hot Chips*, 2019.

[3] J. Gómez-Luna *et al.*, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, 2022.

[4] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, 2005.